

Assignment 5: Build an Action-Oriented Agentic Tutor with SWE-Agent

Course: Software Modeling and Analysis

Group 16

Tiago Cardoso, Palina Kazlova, Igor Jakovlev
School of Software Engineering, Tongji University

May 2026

1 Introduction

The purpose of this assignment was to build an Action-Oriented Agentic Tutor using AI-assisted development tools.

During the design phase, we used ChatGPT to refine ideas and explore the system architecture. Its suggestions were discussed within the team and used to support design decisions.

For implementation, we chose Windsurf instead of Cursor to evaluate whether a lower-cost tool could still support building a basic MVP. It was used to generate parts of the application and assist development. The results are discussed in the final section. The code is uploaded on [github](#).

2 Architecture

This section describes in detail the architecture we agreed on in the team at the end of the design phase. The final product that we received from the coding agent might differ in some details.

2.1 Exercise and labeling creation

The system is organized around course-specific topic labels, which are used as the basis for learner analysis.

The workflow starts when the professor uploads exam questions or individual practice exercises. Exam answers are graded manually, while individual exercises are graded automatically after student submission.

When a new question is added, the system sends it to the LLM together with the existing course topic labels. The LLM suggests matching labels for the topics like "derivatives" or "multiplication" or proposes new ones. A single exercise can have multiple topic labels. The professor can then approve, edit, or create labels manually.

These labels are central to the system, as they connect exercises, exam results, and study recommendations, allowing the tutor to track progress and identify weak topics.

2.2 Study plan creation

The second main workflow is the creation and updating of the student's study plan.

Using the student's exercise logs and exercise metadata, the system checks whether the stored label strengths are outdated, this happens when new exercises are added or when the student completes new exercises. This reduces unnecessary LLM calls, since updates only happen when relevant data changes.

The label strength table stores the result of the LLM analysis over the student's learning history. It represents the estimated performance of the student for each label based on recent exercise activity and predefined evaluation rules.

These rules consider factors such as:

- exercise difficulty,
- correctness of answers,
- recency of attempts,
- and whether the exercise was part of an exam or individual practice.

More recent results have greater influence, and exam questions are given higher importance because they are considered more representative of the student’s current understanding. Some edge cases must also be handled explicitly. For example, if a student has not yet attempted any exercise related to a label, the score is set to 0 by rule. Further similar constraints can be introduced to improve consistency and reduce unstable outputs.

The LLM returns a structured JSON response containing:

- a score for each label (0–100),
- a trend indicator (e.g. improving or declining),
- and a short reasoning field explaining the result.

A score of 0 represents an unseen or very weak topic, while 100 indicates that all associated exercises have been completed successfully.

Once the student’s label strengths are updated, the system generates a study plan in a second LLM enhanced operation. A ”study plan” internally takes the form of a list of ”study blocks”, each associated with a topic from the label strengths list and a list of recommended exercises for that subject. This is based mainly on the weaker labels, using the previously computed scores and reasoning. In this stage, the LLM receives fewer strict instructions. The goal is to allow some variability in the study plan so that recommendations are not always identical. The main fixed constraints are the number of study blocks and exercises selected. We chose:

- 3 to 5 topic blocks
- 2 to 4 exercises per topic

This creates a balanced plan and avoids overloading the student with only one weak topic.

Every night at midnight, a scheduled job checks which students have outdated study plans and runs the update workflow for them. This task is not time-critical, so it can be processed asynchronously using lower-cost infrastructure such as spot instances. Since the study plans are precomputed before students access them, the system avoids unnecessary latency during normal use while reducing operational cost.

Students can also trigger an update themselves through the interface, with a 30-minute cool down. This is useful during exam periods, when a student may complete their plan faster and request a new one immediately.

2.3 Nudge Generation

To reduce hallucination and avoid inappropriate repeated messaging, we decided not to use the LLM to create nudges directly.

Instead, nudges are generated deterministically from the existing study plan. The system selects one exercise from the highest-priority study block and builds a message using predefined templates.

Examples include:

- “You recently improved in recursion. Keep practicing with this next exercise.”
- “90% of students do not solve this on the first try. Can you?”

This ensures that nudges remain concise, actionable, and predictable. The message is shown as a notification when the student opens the selected course. The student can click on said notification to be taken to the study plan. Since study plans are already precomputed, nudge generation is immediate.

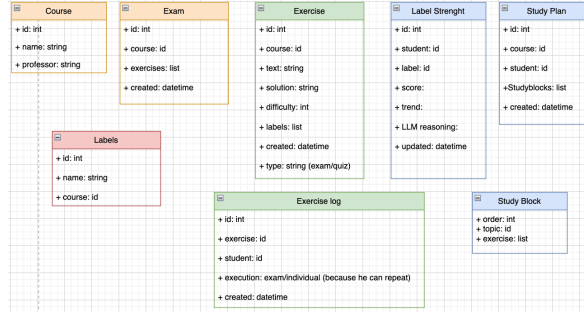


Figure 1: Data Base Schema before AI review

2.4 Supporting Website Features

We chose a minimal, lightweight website UI for this project. The website provides two views of the system, from the viewpoint of the professor and that of a student. They both are accessible by a corresponding student or professor login.

For exams, professors can grade student answers directly in the platform. For individual practice exercises, students can submit answers through the system, and the backend compares them with the stored solution.

The correct solution is provided by the professor when the exercise is created. This means exercise evaluation is deterministic and does not depend on LLM-generated answers, reducing the risk of incorrect feedback.

2.5 Database Schema

After discussing the system design, we manually created an initial database mockup. This was then provided to ChatGPT to convert into SQLite table definitions.

One observation was that ChatGPT identified an inconsistency in our design: several tables referenced the student id, but no student table had been defined. It correctly suggested adding a student table to normalize these references. This was useful because it showed that the model could infer missing structural relationships from context rather than only translating the diagram literally.

At the same time, it failed to identify another issue: the exercise log table initially did not include any field for storing the exercise result or score. Considering that the system depends on student performance data for analysis and study plan generation, this omission should have been noticeable. This highlighted one limitation we observed: while the model can identify some structural inconsistencies, it may overlook domain-specific requirements unless they are stated explicitly, where human are key.

3 Architecture Evaluation

For the system design phase, we used ChatGPT mainly to explore architectural alternatives and understand trade-offs.

We observed that it was especially useful in the early stages, when refining the overall concept and identifying missing relationships in the architecture. However, as the discussion became more detailed, the responses became less reliable and sometimes introduced incorrect assumptions or unnecessary complexity.

One recurring issue was that the model often reinforced our ideas too positively. Instead of critically evaluating them, it frequently focused on validating the proposed solution, which made it harder to identify weaknesses. Because of this, we changed our prompting strategy. Instead of asking open-ended questions, we explicitly asked it to focus only on possible problems, edge cases, and implementation flaws.

This generally produced better results. With more specific prompts, the model was able to identify several overlooked cases and inconsistencies. Some of the suggested problems were unrealistic, but many revealed useful edge cases that we had not considered. This was helpful because it reduced the time spent searching for possible issues and allowed us to focus more on evaluating and solving them.

ChatGPT warned us that the proposed architecture still has some theoretical limitations, mainly related to AI reliability. One example is incorrect topic labeling, where the LLM may assign near-synonymous labels to different exercises, leading to an explosion or over-complication of the labels over time. This would affect later analysis and study plan generation. However, we were actually positively surprised by the quality of label suggestions in our tests. In a simple mathematics course context, the generated labels matched most of the ones we expected.

Another limitation could be that recommendation quality depends heavily on the rules used for evaluating exercise performance. Not all exercises have the same importance or difficulty, so these weighting rules need to be designed carefully. Poor weighting could lead to weak study plans even when the LLM analysis itself is correct.

4 Vibe-Coding

The full architecture was considered too detailed to implement directly through a coding agent without introducing many errors. Because of this, we reduced the scope and removed some optimizations from the MVP. The main goal became evaluating how well the agent could follow structured instructions and integrate Google Gemini API with minimal manual effort.

At first, we tried to use ChatGPT to craft us a good prompt for Windsurf, however we discarded it due to not being satisfied with the quality and crafted a prompt by hand. While not providing a complete specification, we gave the agent a high-level description of the application, including the expected webpages, backend features, and overall workflow. The database schema was also provided with the prompt. This was an important constraint, as it reduced hallucinations and helped keep the generated code aligned with the intended data flow.

5 Code Evaluation

The agent took approximately three hours to produce the first working version. Several permissions still required manual approval, so fully autonomous development was not possible.

Overall, Windsurf performed better than expected and implemented most of the requested functionality without highly detailed prompts.

One observation was that it introduced features beyond our instructions. For example, it used Google Gemini API to generate the nudge message, even though this was not part of the design.

The main difficulty was making it use the correct library versions and Gemini model. We suspect that it often relied on outdated documentation and deprecated APIs. It also ignored some explicit requirements, such as implementing exercise grading as a 0–100 score instead of only “Correct” or “Incorrect”.

In terms of maintainability, debugging often relied on crashes or network inspection because the application lacked clear error messages. However, once a problem was described, the agent was usually able to update the database and adapt related code consistently.

Windsurf produced acceptable code quality, but speed was its main limitation. Long response times became a bottleneck during development. Compared to tools such as Cursor, it appeared slower for iterative work, likely because we used the SWE-1.6 Slow model.

This showed that tool selection matters when moving beyond a prototype. Windsurf was sufficient for building the MVP, but for future work we would likely use a faster model to improve iteration speed.

6 Conclusion

To conclude, we were satisfied with both AI tools used in the project.

ChatGPT was useful mainly during planning. It helped us identify problems earlier, refine workflows, and think through architectural decisions. Although some responses were inaccurate, it still reduced time spent on early design discussions.

Windsurf produced good results for building the prototype. It implemented most of the requested functionality, but its main limitation was speed, which slowed down iteration during development.

The final MVP did not fully match the architecture we originally planned, but it showed that agentic tools can generate a usable prototype when given clear prompts and constraints.

A relevant observation was the time distribution: around 80% of the project was spent discussing features and architecture, while only 20% was spent coding. This suggests that with these tools, more effort shifts toward defining problems clearly and validating solutions, while repetitive implementation work can be delegated to the agent.

7 AI Disclosure

We made use of AI tools for the following tasks:

- Chat GPT - overall planning.
- Windsurf(swe-1.6 Slow) - agentic IDE for vibe-coding.
- Gemini - backend LLM prompt execution.